

Praktikum 1 — Pengenalan Big Data & Lingkungan Kerja, serta Pengolahan Dataset Big Data Menggunakan Python

Modul praktikum berbasis *learning by doing*. Setiap konsep teori langsung diikuti aktivitas praktik pada Google Colab. Mahasiswa dapat menyelesaikan seluruh langkah secara mandiri tanpa pendampingan terus-menerus dari dosen.

A. Identitas Praktikum

Mata Kuliah	Praktikum Big Data
Program Studi / Semester	S1 Informatika / Semester V
Judul Praktikum	Pengenalan Big Data & lingkungan kerja; pengolahan dataset skala besar dengan Python
Pertemuan	Praktikum 1
Durasi	120 menit (di laboratorium) + tugas mandiri
Platform	Google Colab (runtime CPU, tanpa GPU)
Dosen	Afdhal Dinilhak & Ajeng Dwi Asti
Hasil Akhir yang Dikumpulkan	(1) Notebook <code>.ipynb</code> berisi seluruh sel yang sudah dijalankan; (2) file hasil agregasi <code>agregasi_per_jam.csv</code> dan <code>.parquet</code> ; (3) laporan praktikum (PDF)

B. Deskripsi Praktikum

Praktikum ini adalah pintu masuk mata kuliah Big Data. Mahasiswa tidak hanya membaca definisi Big Data, tetapi langsung membuktikan sendiri kapan sebuah dataset mulai “terasa besar”: kapan RAM habis, kapan pembacaan file menjadi lambat, dan strategi apa yang membuat pekerjaan tetap mungkin dilakukan pada satu mesin.

Aktivitas dibagi menjadi dua blok. Blok 1 (± 45 menit) — menyiapkan dan memverifikasi lingkungan kerja: Google Colab, versi Python, *library*, kapasitas RAM/disk, penyimpanan Google Drive, serta pengukuran waktu dan memori. Blok 2 (± 65 menit) — mengolah dataset publik berukuran puluhan MB dengan jutaan baris menggunakan pandas (pemilihan kolom, optimasi tipe data, *chunking*, agregasi), lalu membandingkan pendekatan yang sama menggunakan PySpark dalam mode *local*.

C. Capaian Pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

Kode	Rumusan capaian
CPMK-1	Menjelaskan karakteristik Big Data (5V) dan membedakan masalah data biasa dengan masalah Big Data berdasarkan bukti pengukuran, bukan asumsi.
CPMK-2	Menyiapkan dan memverifikasi lingkungan kerja Big Data berbasis notebook (Colab, pustaka, penyimpanan, sumber daya).
CPMK-3	Memuat, membersihkan, dan mengagregasi dataset berukuran jutaan baris menggunakan Python dengan strategi hemat memori.
CPMK-4	Menganalisis konsumsi waktu dan memori dari beberapa strategi pengolahan, lalu memilih strategi yang tepat beserta alasannya.
CPMK-5	Mengkomunikasikan hasil pengolahan data secara ringkas, jujur, dan dapat direproduksi orang lain.

D. Tujuan Praktikum

1. Mahasiswa dapat menyiapkan notebook Colab yang siap pakai dan mendokumentasikan spesifikasi lingkungannya (Python version, library version, RAM, disk).
2. Mahasiswa dapat mengunduh dataset publik secara terprogram dan memeriksa integritas awalnya (ukuran file, jumlah baris, skema kolom).
3. Mahasiswa dapat mengukur pemakaian memori sebuah `DataFrame` dan menurunkannya melalui pemilihan kolom serta optimasi tipe data.
4. Mahasiswa dapat memproses file yang lebih besar dari RAM yang tersedia menggunakan teknik *chunking*.
5. Mahasiswa dapat menghasilkan tabel agregasi (per jam, per metode pembayaran) dan menyimpannya dalam format CSV serta Parquet.
6. Mahasiswa dapat menjalankan kueri agregasi menggunakan PySpark *local mode* dan menjelaskan perbedaannya dengan pandas.

E. Kompetensi Awal

Prasyarat yang diasumsikan sudah dikuasai sebelum praktikum. Jika salah satu belum, kerjakan bacaan penyegaran pada kolom terakhir sebelum masuk laboratorium.

Kompetensi	Indikator minimal	Penyegaran
Pemrograman Python dasar	Tipe data, <code>list/dict</code> , fungsi, <code>for loop</code> penanganan <i>exception</i> sederhana	McKinney (2022) ch. 2-3

Manipulasi data tabular	Membaca CSV, <i>filtering</i> baris, <code>groupby</code> sederhana dengan <code>pandas</code>	McKinney (2022) ch. 5 & 10
Basis data & SQL	Menulis <code>SELECT ... GROUP BY ... ORDER BY</code>	Materi mata kuliah basis data
Sistem operasi / terminal	Memahami <i>path</i> , perintah <code>ls</code> <code>df</code> <code>free</code>	Operating Systems course material
Akun Google aktif	Dapat membuka Google Colab dan Google Drive (kuota bebas ≥ 2 GB)	-

F. Dasar Teori

F.1 Apa yang membuat data disebut “Big”

Big Data bukan sekadar “data yang banyak”. Definisi operasional yang paling berguna bagi praktisi: data disebut besar ketika cara pengolahan konvensional pada satu mesin sudah tidak memadai lagi – baik karena tidak muat di memori, terlalu lama diproses, terlalu cepat datangnya, atau terlalu beragam bentuknya. Batasnya relatif terhadap sumber daya, sehingga selalu harus *diukur*, bukan diperdebatkan. Karakteristik klasik dirangkum sebagai 5V

Dimensi	Makna	Bukti pada praktikum ini
Volume	Ukuran data melebihi kapasitas nyaman satu mesin	Jutaan baris perjalanan taksi; RAM Colab terbatas (L-4, L-5)
Velocity	Kecepatan data masuk dan kebutuhan latensi hasil	Dataset dipublikasikan bulanan; dibahas konseptual, dipraktikkan pada Praktikum berikutnya
Variety	Terstruktur, semi-terstruktur, tak terstruktur	Perbandingan CSV (baris) vs Parquet (kolom) pada L-7 dan L-9
Veracity	Derajat kepercayaan/kualitas data	Baris anomali: zero distance, durasi negatif, tarif minus (L-6)
Value	Nilai bisnis yang dihasilkan dari data	Tabel pola permintaan per jam sebagai dasar keputusan (L-10, O)

F.2 Tiga tingkatan skala dan alat yang sesuai

Sebelum memilih teknologi, tentukan dulu posisi data. Pendekatan praktis:

- **Muat di memori** (data < $\pm \frac{1}{3}$ RAM) → pandas/NumPy biasa. Paling sederhana dan paling cepat untuk dikembangkan.
- **Lebih besar dari memori, masih satu mesin** → *chunking*, columnar formats (Parquet), pemilihan kolom, *narrow dtypes*, atau *out-of-core engine* (Dask, Polars, DuckDB).
- **Lebih besar dari satu mesin** → *distributed processing* (Apache Spark, Hadoop MapReduce) *over distributed storage* (HDFS, object storage).

Kesalahan umum adalah melompat langsung ke Spark untuk data 200 MB. Spark menambah *overhead* penjadwalan dan serialisasi; pada data kecil ia justru lebih lambat daripada pandas.

F.3 Ekosistem Big Data secara ringkas

Empat lapisan yang perlu dikenali sejak awal: penyimpanan (HDFS, Amazon S3, Google Cloud Storage), pemrosesan (MapReduce, Spark, Flink), orkestrasi & sumber daya (YARN, Kubernetes, Airflow), dan akses/analitik (Hive, Presto/Trino, notebook). Prinsip yang menyatukan semuanya adalah *data locality*: memindahkan komputasi ke tempat data berada jauh lebih murah daripada memindahkan data ke tempat komputasi.

F.4 Format baris vs format kolom

CSV menyimpan data per baris, sehingga membaca satu kolom tetap memerlukan pemindaian seluruh isi file, dan setiap nilai harus diurai dari teks. **Parquet** menyimpan data per kolom dengan skema dan kompresi, sehingga hanya kolom yang diminta yang dibaca. Karena itu, dalam pekerjaan analitik, format columnar biasanya jauh lebih efisien – dan inilah yang akan kita ukur sendiri pada Bagian L-7.

F.5 Lingkungan kerja: mengapa notebook di cloud

Google Colab memberi setiap mahasiswa mesin virtual Linux dengan Python dan pustaka analitik yang sudah terpasang, tanpa proses instalasi lokal. Konsekuensinya harus dipahami: mesin bersifat sementara (*ephemeral*) – ketika *runtime* berakhir atau *idle* terlalu lama, seluruh file di `/content` hilang. Karena itu semua hasil penting disimpan ke Google Drive, dan setiap notebook harus dapat menyalin ulang datanya sendiri dari awal (prinsip *reproducibility*).

G. Tools dan Software

Tool	Versi rekomendasi	Fungsi & catatan
Google Colab	Layanan daring (selalu terbaru)	Lingkungan notebook utama. Gunakan runtime <i>CPU</i> ; GPU tidak dibutuhkan.
Python	3.10 – 3.12	Sudah tersedia di Colab. Jangan mengubah versi Python bawaan Colab.
pandas	≥ 2.0 (idealnya ≥ 2.2)	Pengolahan tabular utama. Bergantung versi: parameter <code>dtype_backend="pyarrow"</code> hanya ada pada pandas ≥ 2.0 .
PyArrow	≥ 14	<i>Engine</i> baca/tulis Parquet. Jika hilang, <code>read_parquet</code> gagal.

Tool	Versi rekomendasi	Fungsi & catatan
PySpark	3.5.x (uji: 3.5.1)	Bergantung versi: Spark 3.5 membutuhkan Java 8/11/17. Spark 4.x menuntut Java 17+ dan berbeda perilaku; jangan campur versi dalam satu kelas.
OpenJDK	11 atau 17	Dibutuhkan PySpark. Biasanya sudah ada di Colab — periksa dengan <code>!java -version</code> .
matplotlib	≥ 3.7	Visualisasi sederhana hasil agregasi.
psutil	≥ 5.9	Mengukur pemakaian memori proses.
Google Drive	—	Menyimpan notebook dan artefak hasil agar tidak hilang saat runtime berakhir.

CATATAN KETERGANTUNGAN VERSI

Colab memperbarui pustaka bawaannya tanpa pemberitahuan. Karena itu Bagian J-1 mewajibkan kita *mencetak* versi yang benar-benar terpasang dan menempelkannya di laporan. Jika sebuah sel gagal, langkah diagnosis pertama selalu: bandingkan versi Anda dengan tabel di atas.

H. Dataset

Dataset utama — NYC TLC Yellow Taxi Trip Records. Data perjalanan taksi kuning New York City yang dipublikasikan resmi oleh NYC Taxi & Limousine Commission, satu file Parquet per bulan, berisi beberapa juta baris. Dataset ini dipilih karena gratis dan legal digunakan, berukuran realistis (puluhan MB terkompresi, ratusan MB di memori), skemanya kaya (waktu, jarak, tarif, pembayaran), dan mengandung anomali nyata sehingga aspek *veracity* dapat dilatih.

- Halaman resmi: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page> (berisi juga kamus *data/data dictionary*)
- File yang dipakai: `yellow_tripdata_2023-01.parquet`
- Kolom yang akan digunakan: `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `passenger_count`, `trip_distance`, `payment_type`, `fare_amount`, `tip_amount`, `total_amount`

Dataset cadangan. Jika jaringan di laboratorium memblokir unduhan dari internet atau tautan berubah, gunakan mekanisme cadangan pada Bagian J-4b yang *membangkitkan* dataset sintesis 5 juta baris secara lokal. Seluruh langkah selanjutnya berlaku sama; hanya angka hasilnya yang berbeda.

ETIKA & SUMBER DATA

Selalu cantumkan sumber, tanggal unduh, dan lisensi dataset di laporan Anda. Jumlah baris tiap bulan berbeda dan penerbit sesekali merevisi file historis — karena itu jangan menyalin angka dari modul, laporkan angka dari eksekusi sendiri.

I. Persiapan Praktikum

Kerjakan seluruh daftar ini sebelum jam praktikum dimulai. Perkiraan waktu: 10 menit.

1. Login ke akun Google, buka colab.research.google.com, buat notebook baru.
2. Ubah nama notebook menjadi **BD_<Kelas>_P01_<NIM>_<Nama>.ipynb**
3. Pastikan runtime bertipe CPU: menu *Runtime* → *Change runtime type* → *Hardware accelerator: None*. GPU tidak diperlukan dan hanya memperbesar antrean.
4. Buat folder di Google Drive: **MyDrive/BigData/Praktikum1/**
5. Pastikan kuota Drive bebas minimal 2 GB.
6. Buat github repository baru dengan format nama: **BigData26_<Kelas>_<NIM>_<Nama>** dan di dalamnya buat folder sesuai urutan pertemuan/praktikum. Contoh: **Praktikum01**
7. Baca Bagian F (Dasar Teori) sampai selesai.

Konvensi notebook. Setiap langkah pada Bagian L = satu sel kode, didahului satu sel teks (*Markdown*) berisi judul langkah dan satu-dua kalimat Anda sendiri. Notebook tanpa sel teks dinilai tidak lengkap.

J. Tutorial dan Langkah Kerja

Jalankan sel secara berurutan dan jangan melewati langkah.

J-1. Verifikasi lingkungan kerja

Tujuan: mengetahui persis mesin apa yang dipakai. Semua kesimpulan performa nanti hanya berlaku untuk spesifikasi ini.

```
import sys, platform, multiprocessing

print("Python :", sys.version.split()[0])
print("OS      :", platform.platform())
print("CPU core:", multiprocessing.cpu_count())

for nama in ["pandas", "numpy", "pyarrow", "matplotlib"]:
    try:
        mod = __import__(nama)
        print(f"{nama:11s}: {mod.__version__}")
    except ImportError:
        print(f"{nama:11s}: BELUM TERPASANG")
```

```
!free -h          # kapasitas dan sisa RAM
!df -h /content   # kapasitas dan sisa disk runtime
!java -version    # dibutuhkan PySpark pada langkah K-8
```

Contoh *output* (nilai aktual pada setiap mesin hampir pasti berbeda):

```

Python   : 3.11.x
OS       : Linux-x.x.x-x86_64-with-glibc2.xx
CPU core: 2
pandas   : 2.x.x
numpy    : 1.x.x / 2.x.x
pyarrow  : 1x.x.x

```

Catat di laporan: versi Python, versi pandas, jumlah core, total RAM, sisa disk.

J-2. Menghubungkan Google Drive dan menyiapkan struktur folder

```

from google.colab import drive
drive.mount("/content/drive")          # ikuti dialog izin yang muncul

import os
DIR_KERJA = "/content/data"            # sementara, cepat
DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum1"  # permanen
os.makedirs(DIR_KERJA, exist_ok=True)
os.makedirs(DIR_SIMPAN, exist_ok=True)
print(os.listdir(DIR_SIMPAN))

```

J-3. Menyiapkan alat ukur waktu dan memori

```

import time, os, psutil

proses = psutil.Process(os.getpid())
catatan = []          # menyimpan hasil pengukuran untuk dibandingkan nanti

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    """Menjalankan fungsi tanpa argumen, mencatat durasi dan pertumbuhan memori."""
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    delta = rss_mb() - m0
    catatan.append({"langkah": label, "detik": round(detik, 2),
                    "delta_rss_mb": round(delta, 1)})
    print(f"[{label}] {detik:.2f} s | RSS {delta:+.1f} MB")
    return hasil

```

Praktikum ini menuntut bukti angka. RSS (*Resident Set Size*) adalah memori fisik yang benar-benar dipakai proses. Angkanya bisa naik-turun karena *garbage collector*, jadi bacalah sebagai indikasi besaran, bukan nilai eksak.

J-4. Mengunduh dataset dan inspeksi awal

```
import urllib.request

URL = ("https://d37ci6vzurychx.cloudfront.net/trip-data/"
      "yellow_tripdata_2023-01.parquet")
PATH = os.path.join(DIR_KERJA, "yellow_tripdata_2023-01.parquet")

if not os.path.exists(PATH):
    ukur("unduh dataset", lambda: urllib.request.urlretrieve(URL, PATH))

print("Ukuran file:", round(os.path.getsize(PATH) / 1024*2, 1), "MB")
```

```
import pyarrow.parquet as pq

meta = pq.ParquetFile(PATH).metadata
print("Jumlah baris  :", f"{meta.num_rows:,}")
print("Jumlah kolom  :", meta.num_columns)
print("Row group     :", meta.num_row_groups)
print("Nama kolom    :", pq.ParquetFile(PATH).schema.names)
```

Perhatikan: metadata di atas terbaca tanpa memuat satu baris pun ke memori. Inilah keunggulan pertama format kolumnar — Data bisa dikenali sebelum memutuskan cara memuatnya.

J-4b. Jalur cadangan bila unduhan gagal

```
import numpy as np, pandas as pd

def bangkitkan_sintetis(n=5_000_000, seed=42):
    rng = np.random.default_rng(seed)
    awal = pd.Timestamp("2023-01-01")
    pickup = awal + pd.to_timedelta(rng.integers(0, 31*24*60, n), unit="m")
    durasi = rng.integers(1, 90, n)
    return pd.DataFrame({
        "tpep_pickup_datetime": pickup,
        "tpep_dropoff_datetime": pickup + pd.to_timedelta(durasi, unit="m"),
        "passenger_count": rng.integers(1, 5, n).astype("float64"),
        "trip_distance": np.round(rng.gamma(2.0, 1.6, n), 2),
        "payment_type": rng.integers(1, 5, n).astype("int64"),
        "fare_amount": np.round(rng.gamma(4.0, 3.0, n), 2),
        "tip_amount": np.round(rng.gamma(1.5, 1.2, n), 2),
    }).assign(total_amount=lambda d: (d.fare_amount + d.tip_amount + 3.0).round(2))

if not os.path.exists(PATH):
    bangkitkan_sintetis().to_parquet(PATH, index=False)
    print("Dataset sintetis dibuat di", PATH)
```

Jika memakai jalur ini, tulis jelas di laporan: “dataset sintetis, bukan data nyata NYC TLC”. Interpretasi pola (mis. jam sibuk) tidak berlaku pada data sintetis karena distribusinya acak seragam.

J-5. Memuat data dan mengukur biaya memorinya

Lakukan dua percobaan dan bandingkan angkanya sendiri: memuat *semua* kolom, lalu memuat *hanya* kolom yang dibutuhkan

```
import pandas as pd

KOLOM = ["tpep_pickup_datetime", "tpep_dropoff_datetime",
         "passenger_count", "trip_distance", "payment_type",
         "fare_amount", "tip_amount", "total_amount"]

penuh = ukur("baca semua kolom", lambda: pd.read_parquet(PATH))
print("Dimensi:", penuh.shape)
print("Memori :", round(penuh.memory_usage(deep=True).sum() / 1024**2, 1), "MB")

del penuh # bebaskan memori sebelum percobaan kedua
import gc; gc.collect()

df = ukur("baca kolom terpilih", lambda: pd.read_parquet(PATH, columns=KOLOM))
print("Dimensi:", df.shape)
print("Memori :", round(df.memory_usage(deep=True).sum() / 1024**2, 1), "MB")
df.head()
```

```
df.info(memory_usage="deep")
df.memory_usage(deep=True).sort_values(ascending=False) / 1024**2 # MB per kolom
```

Selanjutnya turunkan pemakaian memori dengan tipe data yang lebih sempit:

```
sebelum = df.memory_usage(deep=True).sum() / 1024**2

df["passenger_count"] = pd.to_numeric(df["passenger_count"], downcast="float")
df["payment_type"] = df["payment_type"].astype("int8").astype("category")
for kol in ["trip_distance", "fare_amount", "tip_amount", "total_amount"]:
    df[kol] = pd.to_numeric(df[kol], downcast="float")

sesudah = df.memory_usage(deep=True).sum() / 1024**2
print(f"{sebelum:.1f} MB -> {sesudah:.1f} MB (hemat {100*(1-sesudah/sebelum):.1f}%)")
df.dtypes
```

Bergantung versi. Pada pandas ≥ 2.0 Anda juga dapat mencoba `pd.read_parquet(PATH, columns=COLS, dtype_backend="pyarrow")`. Hasil penghematannya berbeda dari cara di atas; jika Anda mencobanya, laporkan keduanya sebagai dua pengukuran terpisah.

J-6. Pemeriksaan kualitas data dan pembersihan (veracity)

```
df["durasi_menit"] = (df["tpep_dropoff_datetime"] -
df["tpep_pickup_datetime"]).dt.total_seconds() / 60

print(df[["trip_distance", "durasi_menit", "total_amount"]].describe())
print("\nNilai kosong per kolom:\n", df.isna().sum())

anomali = {
    "jarak <= 0"           : (df["trip_distance"] <= 0).sum(),
    "durasi <= 0"          : (df["durasi_menit"] <= 0).sum(),
    "durasi > 180 menit"   : (df["durasi_menit"] > 180).sum(),
    "total_amount <= 0"   : (df["total_amount"] <= 0).sum(),
}
for k, v in anomali.items():
    print(f"{k:22s}: {v:,} baris ({100*v/len(df):.2f}%)")
```

```
layak = (
    (df["trip_distance"] > 0) & (df["trip_distance"] < 100) &
    df["durasi_menit"].between(1, 180) &
    (df["total_amount"] > 0)
)
bersih = df.loc[layak].copy()
print(f"{len(df):,} baris -> {len(bersih):,} baris (dibuang {len(df)-len(bersih):,})")
```

Setiap nilai *threshold* di atas adalah keputusan analitis, bukan kebenaran mutlak. Di laporan, tuliskan alasan setiap *threshold* (mis. “durasi > 180 menit dianggap sebagai *error recording*”) – inilah bentuk pertanggungjawaban ilmiah dalam pekerjaan data.

J-7. Memproses file lebih besar dari memori dengan chunking

Simulasikan kondisi klasik “file terlalu besar”: tulis ulang data sebagai CSV (jauh lebih besar karena berupa teks tanpa kompresi), kemudian proseslah bagian demi bagian tanpa pernah memuatnya secara utuh.

```
CSV = os.path.join(DIR_KERJA, "trips.csv")
ukur("tulis CSV", lambda: bersih.to_csv(CSV, index=False))

print("Parquet:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")
print("CSV      :", round(os.path.getsize(CSV) / 1024**2, 1), "MB")
```

```
def agregasi_bertahap(path, ukuran_chunk=500_000):
    jumlah_baris = 0
    total_nilai = 0.0
    per_jam = {}
    potongan = pd.read_csv(
        path,
        usecols=["tpep_pickup_datetime", "total_amount"],
        parse_dates=["tpep_pickup_datetime"],
        chunksize=ukuran_chunk)
    for chunk in potongan:
        jumlah_baris += len(chunk)
        total_nilai += chunk["total_amount"].sum()
        for jam, sub in chunk.groupby(chunk["tpep_pickup_datetime"].dt.hour):
            n, nilai = per_jam.get(jam, (0, 0.0))
            per_jam[jam] = (n + len(sub), nilai + sub["total_amount"].sum())
    return jumlah_baris, total_nilai, per_jam

baris, total, per_jam_chunk = ukur("agregasi chunking", lambda: agregasi_bertahap(CSV))
print(f"{baris:,} baris | rata-rata total_amount = {total/baris:.2f}")
```

Perhatikan kolom `delta_rss_mb` pada hasil `ukur()`: pertumbuhan memori tetap kecil meskipun jumlah baris besar, karena hanya satu potongan berada di memori pada satu waktu. Ini adalah inti gagasan pemrosesan *out-of-core*

J-8. Pendekatan terdistribusi: PySpark local mode

Instalasi berikut diuji dengan PySpark 3.5.1 di atas OpenJDK 11/17. Jika `!java -version` pada L-1 menunjukkan Java 8, gunakan tetap PySpark 3.5.x. Jangan memasang PySpark 4.x pada modul ini.

```
!pip install -q pyspark==3.5.1
!java -version
```

```
from pyspark.sql import SparkSession, functions as F

spark = (SparkSession.builder
        .appName("BD-P01")
        .master("local[*]") # semua core mesin ini
        .config("spark.driver.memory", "4g")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate())
spark.sparkContext.setLogLevel("WARN")
print("Spark:", spark.version)
```

```

sdf = spark.read.parquet(PATH)
sdf.printSchema()
print("Jumlah baris:", ukur("spark count", lambda: sdf.count()))

sdf_bersih = (sdf
    .withColumn(
        "durasi_menit",
        (F.col("tpep_dropoff_datetime").cast("long")
         - F.col("tpep_pickup_datetime").cast("long")) / 60)
    .filter((F.col("trip_distance") > 0) & (F.col("trip_distance") < 100))
    .filter((F.col("durasi_menit") >= 1) & (F.col("durasi_menit") <= 180))
    .filter(F.col("total_amount") > 0))

hasil_spark = (sdf_bersih
    .withColumn("jam", F.hour("tpep_pickup_datetime"))
    .groupBy("jam")
    .agg(F.count("*").alias("jumlah"),
        F.round(F.avg("trip_distance"), 2).alias("rata_jarak"),
        F.round(F.avg("total_amount"), 2).alias("rata_tarif"))
    .orderBy("jam"))

ukur("spark agregasi", lambda: hasil_spark.show(24, truncate=False))

```

```

sdf_bersih.createOrReplaceTempView("trips")
spark.sql("""
    SELECT payment_type,
           COUNT(*)           AS jumlah,
           ROUND(AVG(total_amount), 2) AS rata_tarif,
           ROUND(AVG(tip_amount), 2)  AS rata_tip
    FROM trips
    GROUP BY payment_type
    ORDER BY jumlah DESC
""").show()

hasil_spark.explain()    # rencana eksekusi – perhatikan Exchange / HashAggregate
spark.stop()             # bebaskan sumber daya sebelum lanjut

```

Amati dua hal: (1) `sdf.count()` memakan waktu jauh lebih lama dari yang diduga karena Spark bersifat *lazy* – pekerjaan baru berjalan saat ada *action*; (2) untuk data sebesar ini, pandas kemungkinan besar lebih cepat. Itu bukan kegagalan Spark, melainkan bukti bahwa tools harus dipilih sesuai skala.

J-9. Agregasi akhir dengan pandas dan penyimpanan artefak

```
bersih["jam"] = bersih["tpep_pickup_datetime"].dt.hour

agregasi = (bersih
    .groupby("jam")
    .agg(jumlah_perjalanan=("total_amount", "size"),
        rata_jarak=("trip_distance", "mean"),
        rata_durasi=("durasi_menit", "mean"),
        rata_tarif=("total_amount", "mean"),
        median_tarif=("total_amount", "median"))
    .round(2)
    .reset_index())

agregasi.to_csv(f"{DIR_SIMPAN}/agregasi_per_jam.csv", index=False)
agregasi.to_parquet(f"{DIR_SIMPAN}/agregasi_per_jam.parquet", index=False)
pd.DataFrame(catatan).to_csv(f"{DIR_SIMPAN}/pengukuran_kinerja.csv", index=False)

agregasi
```

Tiga file inilah artefak yang dikumpulkan. Buka kembali folder Drive Anda dan pastikan ketiganya benar-benar ada sebelum menutup runtime.

J-10. Visualisasi hasil

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 3.4))
ax.bar(agregasi["jam"], agregasi["jumlah_perjalanan"])
ax.set_xlabel("Jam penjemputan (0-23)")
ax.set_ylabel("Jumlah perjalanan")
ax.set_title("Distribusi perjalanan per jam")
ax.set_xticks(range(0, 24))
fig.tight_layout()
fig.savefig(f"{DIR_SIMPAN}/distribusi_per_jam.png", dpi=150)
plt.show()
```

Satu grafik, satu pesan. Jangan menumpuk beberapa metrik dengan skala berbeda dalam satu sumbu.

K. Ringkasan Kode/Command

Daftar referensi singkat. Perintah yang diawali dengan ! adalah *shell command* yang dijalankan dari dalam sel notebook.

Kode / Command	Langkah	Kegunaan
<code>!free -h / !df -h</code>	J-1	Melihat kapasitas RAM dan disk runtime
<code>drive.mount("/content/drive")</code>	J-2	Menghubungkan Google Drive ke runtime
<code>psutil.Process().memory_info().rss</code>	J-3	Membaca memori fisik yang dipakai proses
<code>urllib.request.urlretrieve(URL, PATH)</code>	J-4	Mengunduh file dataset secara terprogram
<code>pq.ParquetFile(PATH).metadata</code>	J-4	Membaca skema & jumlah baris tanpa memuat data
<code>pd.read_parquet(PATH, columns=KOLOM)</code>	J-5	Memuat sebagian kolom saja (projection)
<code>df.memory_usage(deep=True)</code>	J-5	Mengukur memori per kolom
<code>pd.to_numeric(s, downcast="float")</code>	J-5	Menyempitkan tipe numerik
<code>s.astype("category")</code>	J-5	Menghemat memori kolom berkardinalitas rendah
<code>df.loc[kondisi].copy()</code>	J-6	Menyaring baris layak analisis
<code>pd.read_csv(..., chunksize=n)</code>	J-7	Membaca file per potongan (out-of-core)
<code>SparkSession.builder.master("local[*]")</code>	J-8	Membuat sesi Spark memakai semua core lokal
<code>sdf.createOrReplaceTempView("trips")</code>	J-8	Mendaftarkan DataFrame agar bisa dikueri SQL
<code>df.explain()</code>	J-8	Melihat rencana eksekusi kueri
<code>df.to_parquet(path)</code>	J-9	Menyimpan hasil dalam format kolumnar

L. Penjelasan Kode/Command

L1 Mengapa `columns=` menghemat sangat banyak

Parquet menyimpan tiap kolom sebagai blok terpisah. Saat Anda meminta 8 dari ±1U kolom, *reader* hanya menyentuh blok yang relevan – *column pruning*. Ini menghemat I/O disk, waktu dekompresi, dan memori sekaligus. Pada CSV hal ini tidak mungkin: seluruh baris tetap harus dibaca dan diurai lebih dulu, sehingga `usecols` hanya menghemat memori, bukan waktu baca.

L2 Tipe data dan biayanya

Secara bawaan pandas memakai `int64/float64` – 8 byte per *value*. Kolom `payment_type` hanya berisi beberapa kode berbeda, sehingga `int8` (1 byte) atau `category` (indeks kecil + kamus nilai) sudah cukup. Untuk 3 juta baris, selisih 7 byte per nilai berarti sekitar 20 MB per kolom. Peringatan penting: `downcast="float"` mengubah `float64` menjadi `float32` dan mengurangi presisi. Untuk uang, ini boleh untuk analisis agregat, tetapi tidak untuk perhitungan tagihan.

L3 Cara kerja `chunksize`

`pd.read_csv(..., chunksize=n)` tidak mengembalikan `DataFrame`, melainkan *iterator*. Setiap iterasi memuat baris, Anda proses, lalu potongan itu dibuang dari memori. Pola ini disebut *streaming*

aggregation. Syaratnya: agregasi harus dapat digabungkan (*associative*) – jumlah, hitungan, minimum, maksimum bisa; median dan nilai unik eksak tidak bisa langsung dan memerlukan teknik khusus (mis. penyimpanan sementara atau algoritma perkiraan). Ini adalah gagasan yang sama yang mendasari MapReduce.

L4 Spark: lazy evaluation, partitions, shuffle

`master("local[*]")` menjalankan Spark dalam satu proses dengan thread sebanyak core yang tersedia – cukup untuk memahami konsepnya tanpa kluster. Transformasi (`filter withColumn groupBy`) hanya membangun sebuah rencana; eksekusi dimulai oleh *action* (`count show write`). `spark.sql.shuffle.partitions` dibuat 8 (bawaan 200) karena data kita kecil: 200 partisi pada 2 core justru menambah *overhead*. Pada keluaran `explain()`, kata `Exchange` menandai *shuffle* perpindahan data antar partisi, operasi termahal dalam pemrosesan terdistribusi.

L5 Mengapa mengukur RSS, bukan hanya `memory_usage`

`df.memory_usage(deep=True)` mengukur memori objek DataFrame itu sendiri. RSS mengukur seluruh proses Python: DataFrame, salinan sementara saat operasi berjalan, file-reader buffers. Selisih keduanya menjelaskan mengapa memuat file 300 MB bisa memerlukan RAM jauh lebih besar dari 300 MB – dan mengapa *runtime* bisa mati (*crash*) meskipun “datanya kelihatannya muat”.

M. Output yang Diharapkan

Yang dinilai adalah bentuk output, bukan angkanya. Nilai aktual bergantung pada dataset, library version, dan spesifikasi runtime – dan memang seharusnya berbeda antar mahasiswa.

Langkah	Bentuk output yang harus muncul
L-1	Daftar versi Python & pustaka; ringkasan RAM, disk, jumlah core; versi Java
L-2	Pesan <i>Mounted at /content/drive</i> dan daftar isi folder tujuan
L-4	Ukuran file dalam MB; jumlah baris berformat ribuan; jumlah kolom; daftar nama kolom
L-5	Dua pasang angka memori (semua kolom vs kolom terpilih) dan satu baris persentase penghematan setelah optimasi tipe
L-6	Tabel <code>describe()</code> , hitungan nilai kosong, empat baris hitungan anomali, dan jumlah baris sebelum→sesudah
L-7	Perbandingan ukuran Parquet vs CSV; waktu agregasi chunking beserta <code>delta_rss_mb</code> yang kecil
L-8	Skema Spark, tabel hasil <code>show()</code> maksimal 24 baris, tabel per <code>payment_type</code> , dan rencana eksekusi <code>explain()</code>
L-9	Tabel agregasi 24 baris × 6 kolom; tiga file tersimpan di Drive
L-10	Satu diagram batang berlabel lengkap (judul, nama sumbu) dan file PNG tersimpan

KEJUJURAN HASIL

Jika sebuah sel gagal, jangan dihapus. Sertakan pesan galat dan tulis satu paragraf diagnosis: apa yang gagal, apa dugaan penyebabnya, apa yang Anda coba. Sel gagal yang didiagnosis dengan baik bernilai lebih tinggi daripada sel “berhasil” yang tidak dipahami.

N. Analisis Hasil

Susun tabel perbandingan dari `pd.DataFrame(records)`, lalu tulis analisis Anda dengan mengacu pada angka Anda sendiri:

1. **Biaya membaca data.** Berapa rasio waktu dan memori antara membaca semua kolom dan membaca kolom terpilih? Apakah rasio itu sebanding dengan rasio jumlah kolom? Jika tidak, mengapa?
2. **Efek optimasi tipe.** Berapa persen memori yang Anda hemat? Kolom mana yang paling besar kontribusinya? Apa risiko yang Anda terima sebagai gantinya?
3. **Format penyimpanan.** Berapa kali lebih besar CSV dibanding Parquet pada data yang sama? Jelaskan dua penyebabnya.
4. **Chunking.** Bandingkan `delta_rss_mb` antara pemuatan penuh dan agregasi chunking. Apa yang Anda korbakan untuk memperoleh hemat memori itu?
5. **pandas vs Spark.** Mana yang lebih cepat pada data ini? Pada ukuran berapa perkiraan Anda urutannya akan berbalik, dan atas dasar apa Anda memperkirakan demikian?
6. **Kualitas data.** Berapa persen baris yang dibuang? Apakah penghapusan itu berpotensi menghasilkan bias sistematis pada kesimpulan (mis. `trip_distance` ikut terbuang)?

Aturan penulisan analisis: satu klaim, satu angka, satu alasan. Hindari kalimat tanpa bukti seperti “Spark lebih cepat karena terdistribusi”.

O. Studi Kasus

Kasus: penjadwalan armada oleh operator taksi

Sebuah operator memiliki 1.200 armada dan ingin menyusun jadwal *shift* pengemudi untuk kuartal berikutnya. Manajemen operasional meminta jawaban atas tiga pertanyaan: (1) pada jam berapa permintaan tertinggi dan terendah; (2) apakah pola tersebut berbeda antara hari kerja dan akhir pekan; (3) apakah perjalanan pada jam sibuk lebih pendek namun lebih menguntungkan per menit dibanding jam lain. Tim data Anda hanya diberi waktu satu hari kerja dan satu mesin – tanpa klaster.

Yang harus Anda kerjakan. Gunakan `bersih` dari J-6, tambahkan dimensi hari, lalu jawab ketiga pertanyaan dengan tabel dan maksimal dua grafik.

```
bersih["hari"] = bersih["tpep_pickup_datetime"].dt.day_name()
bersih["akhir_pekan"] = bersih["tpep_pickup_datetime"].dt.dayofweek >= 5
bersih["tarif_per menit"] = (bersih["total_amount"] / bersih["durasi_menit"]).round(3)

ringkas = (bersih
    .groupby(["akhir_pekan", "jam"])
    .agg(jumlah=("total_amount", "size"),
        rata_durasi=("durasi_menit", "mean"),
        rata_tarif_per_menit=("tarif_per_menit", "mean"))
    .round(2))
ringkas.head(12)
```

Keluaran yang diminta manajemen: tiga rekomendasi operasional, masing-masing satu kalimat, masing-masing menyebut satu angka pendukung dan satu batasan (mis. “data hanya satu bulan musim dingin, pola musiman belum tentu berlaku”).

P. Latihan

Dikerjakan di dalam jam praktikum, langsung di notebook yang sama, setelah J-10 selesai.

- Latihan 1:** Hitung 5 nilai `trip_distance` terbesar pada data bersih dan tampilkan baris lengkapnya. Apakah baris-baris itu masuk akal? Beri komentar satu kalimat.
- Latihan 2:** Ulangi J-7 dengan `chunksize=100_000` lalu `chunksize=1_000_000`. Catat waktu dan `delta_rss_mb` keduanya, dan jelaskan arah *trade-off*-nya.
- Latihan 3:** Buat kolom `persen_tip = tip_amount / fare_amount * 100`, tangani kasus `fare_amount == 0` dengan benar, lalu hitung rata-rata persen tip per `payment_type`. Apa dugaan Anda tentang perbedaan antar metode pembayaran?
- Latihan 4:** Simpan `bersih` sebagai Parquet dengan dua kompresi berbeda (`compression="snappy"` dan `"gzip"`). Bandingkan ukuran file dan waktu tulisnya.
- Latihan 5:** Tulis ulang agregasi per jam J-9 memakai Spark SQL, lalu buktikan bahwa hasil keduanya konsisten dengan membandingkan kolom `jumlah` secara terprogram (mis. selisih maksimum = 0). Jika ada selisih, jelaskan sumbernya.

Q. Tugas Praktikum

Batas waktu: 7 hari setelah praktikum, melalui LMS. Kerjakan mandiri; diskusi dibolehkan, penyalinan tidak.

- Ganti dataset.** Ulangi J-4 sampai J-10 dengan file bulan lain (mis. `yellow_tripdata_2023-07.parquet`) atau dengan dataset *green taxi*. Sesuaikan nama kolom

bila berbeda – periksa dahulu dengan metadata Parquet.

2. **Bandingkan dua bulan.** Sajikan satu tabel perbandingan pola per jam antara bulan praktikum dan bulan pilihan Anda, beserta satu grafik.
3. **Laporan kinerja.** Lampirkan `pengukuran_kinerja.csv` dan bahas: langkah mana yang paling mahal, dan satu perubahan konkret yang Anda usulkan untuk mempercepatnya.
4. **Refleksi teori.** Satu halaman: pada dimensi V mana dataset ini benar-benar “big”, dan pada dimensi mana sebenarnya tidak. Dukung dengan angka dari eksekusi Anda.
5. **Reproducibility.** Pastikan notebook Anda dapat dijalankan dari sel pertama pada runtime baru yang bersih (*Runtime → Restart and run all*) tanpa intervensi manual. Ini bagian dari penilaian.

Push dan Upload semua berkas yang terdiri dari file notebook `.ipynb` (dengan output tersimpan), file CSV, Parquet, dan PNG hasil ke folder **Praktikum01** pada git repository masing-masing. **Laporan praktikum** (format PDF) dikumpulkan melalui iLearn dengan **deadline** pengumpulan tanggal **13 September 2026 pukul 21.00 WIB**.

R. Pertanyaan Evaluasi

1. Jelaskan mengapa “ukuran data” saja tidak cukup untuk menyebut suatu masalah sebagai masalah Big Data. Berikan satu contoh data kecil yang tetap sulit diolah.
2. Dari 5V, dimensi mana yang paling menonjol pada dataset praktikum ini? Dukung jawaban Anda dengan satu angka hasil pengukuran.
3. Sebutkan dua alasan teknis mengapa Parquet lebih kecil dan lebih cepat dibaca daripada CSV untuk beban kerja analitik.
4. Anda punya CSV 40 GB dan mesin dengan RAM 1G GB. Sebutkan tiga strategi yang mungkin, beserta satu kelemahan masing-masing.
5. Apa yang dimaksud *lazy evaluation* pada Spark, dan bagaimana hal itu memengaruhi cara Anda mengukur waktu eksekusi secara benar?
6. 4. Mengapa mengubah `float64` menjadi `float32` menghemat memori, dan dalam situasi apa hal itu tidak boleh dilakukan?
7. Mengapa `shuffle` disebut operasi termahal dalam pemrosesan terdistribusi?
8. Mengapa file di `/content` pada Colab tidak boleh dijadikan tempat penyimpanan akhir, dan apa konsekuensinya bagi desain notebook Anda?

S. Kesimpulan

Praktikum ini menanamkan tiga hal yang akan dipakai sepanjang mata kuliah. **Pertama**, “big” adalah pernyataan tentang hubungan antara data dan sumber daya – karena itu langkah pertama selalu mengukur, bukan menebak. **Kedua**, sebagian besar masalah skala pada tahap awal dapat diselesaikan

tanpa kluster: memilih kolom, memilih tipe data, memilih format penyimpanan, dan memproses per potongan sudah memberi penghematan besar dengan biaya kerumitan yang rendah. **Ketiga**, alat terdistribusi seperti Spark adalah jawaban untuk skala tertentu, dan menerapkannya pada skala yang salah justru memperlambat pekerjaan.

T. Daftar Pustaka

1. McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
2. Chambers, B., & Zaharia, M. (2018). *Spark: The Definitive Guide — Big Data Processing Made Simple* O'Reilly Media.
3. White, T. (2015). *Hadoop: The Definitive Guide* (4th ed.). O'Reilly Media.
4. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
5. Marz, N., & Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications.
6. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107-113.
7. Apache Software Foundation. *Apache Spark Documentation* spark.apache.org/docs/latest
8. Apache Software Foundation. *Apache Parquet Documentation* parquet.apache.org/docs
9. pandas Development Team. *pandas User Guide — Scaling to Large Datasets* [pandas.pydata.org/docs/ user_guide/scale.html](https://pandas.pydata.org/docs/user_guide/scale.html)
10. NYC Taxi & Limousine Commission. *TLC Trip Record Data & Data Dictionary* [ny c.gov/site/tlc/about/tlc-trip-record-data.page](https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page)